三次贝塞尔曲线绘制实验报告

一、实验目的

给定平面内四个任意控制点，使用 C++ 与 OpenGL 绘制三次贝塞尔曲线，并通过控制点拖动观察曲线响应。完成从连续参数曲线到离散图元的转换，以及建立“控制点状态变化—曲线重新求值—画面更新”的完整链路。

二、理论分析

1. 参数方程与三阶形式

三次贝塞尔曲线由四个控制点 $P_0, P_1, P_2, P_3 \in \mathbb{R}^2$ 和参数 $t \in [0,1]$ 确定：

$$B(t) = \sum_{i=0}^{3} P_i B_{i,3}(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3.$$

其中 $B_{i,3}(t)$ 为三次 Bernstein 基函数。写成分量形式为

$$x(t) = \sum_{i=0}^{3} x_i B_{i,3}(t), \qquad y(t) = \sum_{i=0}^{3} y_i B_{i,3}(t).$$

四个控制点对应至多三次的参数多项式，而中间两个控制点并不作为插值条件。与一般的函数图像 $y = f(x)$ 相比，这种表示允许曲线出现竖直切线、回折等情形，也为后续均匀参数采样提供了直接依据。

2. 几何性质及其对实现的约束

对曲线求导可得

$$B'(t) = 3\left[(1-t)^2 (P_1 - P_0) + 2(1-t)t(P_2 - P_1) + t^2 (P_3 - P_2)\right].$$

在参数区间两端，有

$$B(0) = P_0, \qquad B(1) = P_3,$$

$$B'(0) = 3(P_1 - P_0), \qquad B'(1) = 3(P_3 - P_2).$$

这组关系决定了控制点的作用方式：P_0、P_3 直接约束曲线端点，P_1、P_2 则通过两端切方向影响曲线形状。因此，贝塞尔曲线是受控制多边形约束的参数化造型，而不是依次经过四个点的插值曲线。

此外，Bernstein 基函数在 $[0,1]$ 上非负，并由二项式展开可知

$$\sum_{i=0}^{3} B_{i,3}(t) = ((1-t)+t)^3 = 1.$$

所以 $B(t)$ 是控制点的凸组合，整条曲线必然位于控制点凸包内；对控制点施加仿射变换，也等价于对曲线施加同一变换。凸包性为绘制结果提供了独立于视觉观察的判据，仿射不变性则保证了坐标变换不会改变曲线相对于控制多边形的几何关系。

3. de Casteljau 与 Bernstein 求值的方案选择

de Casteljau 算法通过重复线性插值得到同一个曲线点：

$$P_i^{(1)} = (1-t)P_i + tP_{i+1},$$

$$P_i^{(2)} = (1-t)P_i^{(1)} + tP_{i+1}^{(1)}, \qquad B(t) = P_0^{(3)}.$$

将其逐层展开即可得到前述 Bernstein 多项式，因此二者在数学上完全等价，差别只在计算组织方式。

本实验阶数固定为 3，采用展开式直接求值更合适：每个参数点只需计算四个基函数，再进行横、纵坐标的加权求和，不需要递归或额外的中间点数组，算法复杂度和实现路径都很明确。de Casteljau 更适合任意阶曲线、递归细分或强调数值稳定性的场景，但在固定四点的任务中使用它不会带来实际收益。

4. 连续曲线的离散化

OpenGL 最终处理的是有限顶点及图元，连续曲线必须转换为离散几何。程序将参数区间等分为 $N = 200$ 段：

$$t_i = \frac{i}{N}, \qquad i = 0,1,\ldots,N,$$

计算 $B(t_0),\ldots,B(t_{200})$，再用相邻点之间的线段逼近曲线。因此，屏幕上的结果本质上是 200 段弦构成的折线，而不是解析曲线本身。

均匀参数采样实现简单，但不等价于均匀弧长采样：参数步长固定时，曲率较大的区域未必获得更密的几何分辨率。折线逼近误差主要受局部曲率和参数步长共同影响。对本实验的窗口尺度和控制点变化范围，200 个采样区间已经能够获得稳定、平滑的视觉效果；若面向高精度输出，应进一步采用曲率或弦误差驱动的自适应细分，而不是单纯提高固定采样数。

三、从理论到 C++/OpenGL 的实现

1. 整体结构与状态流

实现将数学状态、曲线求值和图形输出分开组织：

```
controlPoints：控制点状态
        │
        ▼
cubicBezier(t)：解析曲线求值
        │
        ▼
display()：离散顶点构造与 OpenGL 图元提交
        │
        ▼
glutSwapBuffers()：显示完整帧

mouse()/motion() 修改控制点 → glutPostRedisplay() → 重新执行上述过程
```

这种组织方式使 `cubicBezier()` 保持为不依赖 OpenGL 的纯计算函数；拖动操作只修改数学模型，不直接处理像素。每次重绘都根据当前控制点重新生成曲线，避免了数学状态与显示缓存不一致的问题。

2. 数据表示与核心求值

```
struct Vec2 {
    float x;
    float y;
};

std::array<Vec2, 4> controlPoints{{
    {100.0f, 500.0f}, {250.0f, 100.0f},
    {550.0f, 100.0f}, {700.0f, 500.0f}
}};

Vec2 cubicBezier(const std::array<Vec2, 4>& points, float t) {
    const float u = 1.0f - t;
    const float b0 = u * u * u;
    const float b1 = 3.0f * u * u * t;
    const float b2 = 3.0f * u * t * t;
    const float b3 = t * t * t;

    return {
        b0 * points[0].x + b1 * points[1].x + b2 * points[2].x + b3 * points[3].x,
        b0 * points[0].y + b1 * points[1].y + b2 * points[2].y + b3 * points[3].y
    };
}
```

`std::array<Vec2, 4>` 将三阶曲线固定需要四个控制点这一约束直接落实到类型层面，避免引入不必要的动态容器。`cubicBezier()` 中的 `u`、`b0...b3` 对应 $(1-t)$ 与四个 Bernstein 系数，返回值的两个表达式分别实现 $x(t)$ 和 $y(t)$。

函数以常量引用接收控制点，避免数组复制并保证求值过程不修改状态。坐标采用 `float`，与 `glVertex2f` 的接口精度一致；在当前坐标范围和屏幕分辨率下，没有必要为求值部分引入更高精度类型。窗口尺寸、采样数和拾取半径则提取为具名常量，避免算法参数散落在渲染与交互代码中。

3. 离散顶点构造与图元提交

曲线顶点在 `display()` 中动态生成：

```
glBegin(GL_LINE_STRIP);
for (int i = 0; i <= kSampleCount; ++i) {
    const float t = static_cast<float>(i) / kSampleCount;
    const Vec2 point = cubicBezier(controlPoints, t);
    glVertex2f(point.x, point.y);
}
glEnd();
```

这里实际构造的是顶点序列 $B(t_0), ..., B(t_N)$。`GL_LINE_STRIP` 仅按提交顺序连接相邻顶点且不自动闭合，与开放曲线段的拓扑一致。控制多边形复用同一图元形式，但其顶点直接来自 `controlPoints`；控制点则使用 `GL_POINTS` 单独绘制。

程序采用的是固定功能管线下的立即模式。`glColor3f()` 和 `glPointSize()` 设置当前图元状态，`glBegin()/glEnd()` 界定图元，`glVertex2f()` 提交二维顶点，最终由 `glutSwapBuffers()` 交换双缓冲。绘制顺序为先控制多边形、再曲线、最后控制点，使辅助几何不遮挡主要观察对象。

这种方案省略了 VBO、着色器和缓冲区管理，适合数据量很小且强调算法本身的实验，但代价是每帧重复求值并产生较多立即模式调用。若扩展为高采样密度或多曲线系统，应将顶点组织改为批量缓冲数据；本实验没有必要为此增加实现复杂度。

4. 坐标映射与渲染一致性

程序通过

```
gluOrtho2D(0, kWindowWidth, kWindowHeight, 0);
```

建立与窗口像素范围一致的正交投影，同时将 y 轴方向设为向下。这样，控制点坐标、OpenGL 绘图坐标和 GLUT 鼠标坐标使用同一套二维坐标约定，拖动过程中无需反复转换 y 坐标。

这一处理的价值不只是减少一次减法，更重要的是统一了数学状态与交互状态的坐标语义。拾取、更新和渲染均使用同一坐标系，避免鼠标位置与曲线位置因坐标翻转而出现系统性偏差。模型观察变换保持单位矩阵，投影只负责二维坐标到显示空间的映射，没有引入与问题无关的三维变换。

5. 交互更新机制

控制点拾取采用圆形邻域：

$$\|(x, y) - P_i\| < kPickRadius, \qquad kPickRadius = 15.$$

代码使用 `std::hypot()` 计算距离，并通过 `draggedPoint` 记录当前拖动对象的索引，`-1` 表示未选中。鼠标按下只负责建立选择状态，移动回调更新对应控制点，释放后清除状态。

`motion()` 更新控制点后调用 `glutPostRedisplay()`，将重绘交回 GLUT 事件循环，而不是在输入回调中直接执行渲染。这样，输入处理与画面生成保持单向关系，也使连续拖动产生的多次状态更

新能够在后续显示回调中统一处理。对于仅含四个控制点、201 个曲线顶点的规模，整帧重新采样的开销很小，采用缓存反而会增加状态同步成本。

四、实验结果与验证

1. 解析验证

初始控制点为

控制点	P_0	P_1	P_2	P_3
坐标	$(100,500)$	$(250,100)$	$(550,100)$	$(700,500)$

取三个具有明确基函数取值的位置进行验证：

- $t = 0$ 时，$B(0) = P_0 = (100,500)$；
- $t = 1$ 时，$B(1) = P_3 = (700,500)$；
- $t = 0.5$ 时，四个基函数分别为 $1/8$、$3/8$、$3/8$、$1/8$，所以

$$B(0.5) = \frac{P_0 + 3P_1 + 3P_2 + P_3}{8} = (400,200).$$

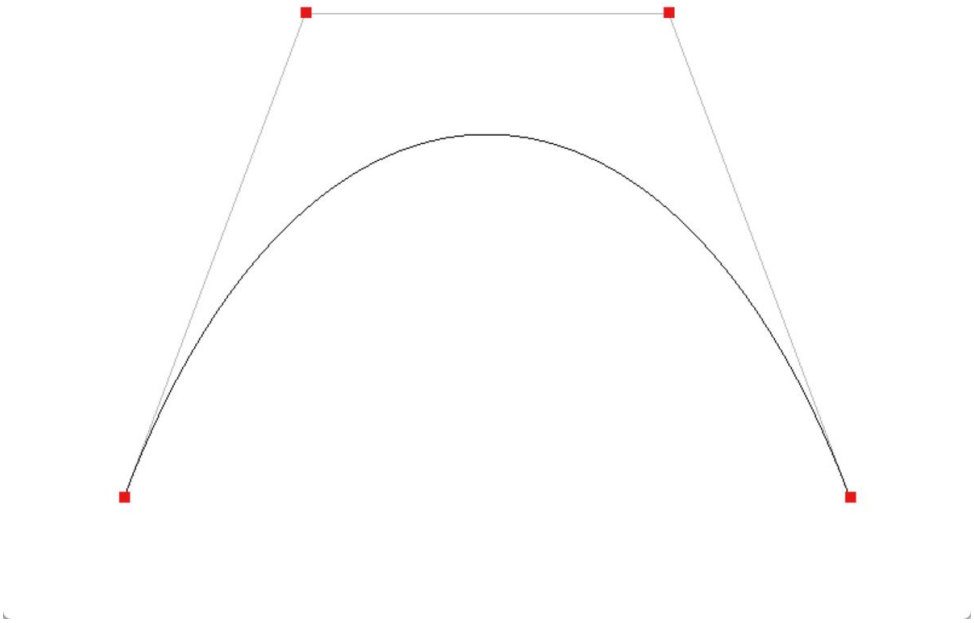
初始控制点关于 $x = 400$ 对称，由 Bernstein 加权的对称性可知曲线也应具有相同对称轴。这一结论与初始运行结果一致。

2. 多组控制点拖动验证

为验证端点约束、切向控制和凸包性，依次调整四个控制点，各阶段坐标如下：

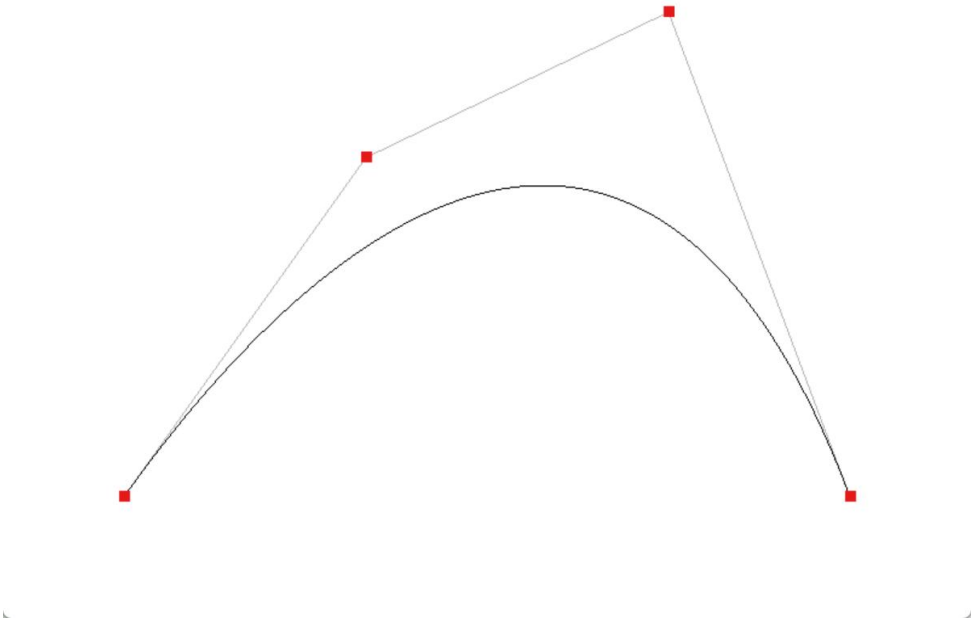
验证状态	P_0	P_1	P_2	P_3
初始状态	$(100,500)$	$(250,100)$	$(550,100)$	$(700,500)$
移动 P_1 后	$(100,500)$	$(300,220)$	$(550,100)$	$(700,500)$
再移动 P_2 后	$(100,500)$	$(300,220)$	$(620,180)$	$(700,500)$
再移动 P_0、P_3 后	$(80,380)$	$(300,220)$	$(620,180)$	$(720,430)$

初始状态下，曲线与控制多边形均呈左右对称：



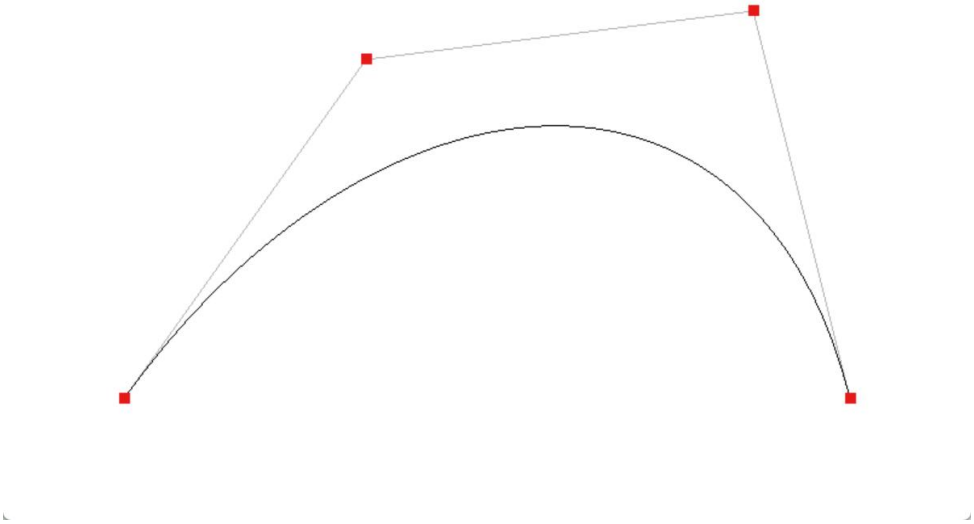
初始状态

将 P_1 移至 (*300,220*) 后，P_0 保持不变，而曲线起始切向量由 $3(P_1 - P_0)$ 决定，因此左半段方向和弯曲程度发生明显变化：



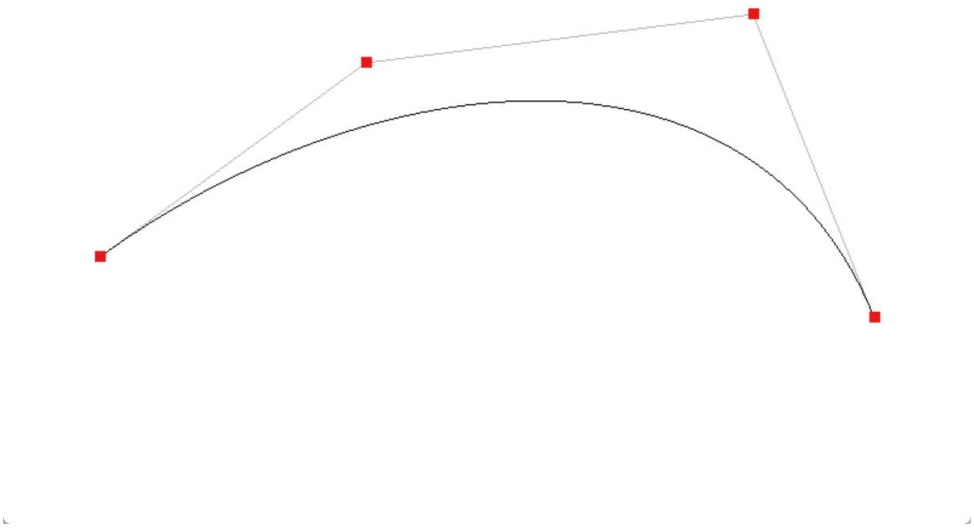
移动 P1 后

继续移动 P_2 后，末端切向量 $3(P_3 - P_2)$ 改变，曲线右半段随控制多边形调整；同时，曲线仍未穿过中间控制点，符合其非插值特性：



移动 P1、P2 后

最后移动 P_0、P_3，曲线两端分别跟随新端点，且整条曲线仍位于更新后的控制点凸包内：



四个控制点均调整后

多组结果表明，程序不仅生成了形式正确的初始曲线，而且在控制点持续变化时仍能保持端点插值、切向响应和凸包约束。截图验证与解析求值结论相互印证。

五、编译与运行

程序依赖 OpenGL、GLU 和 GLUT/freeglut，源文件为 `01.cpp`。

```
# Linux
g++ -std=c++11 01.cpp -o bezier -lGL -lGLU -lglut

# Windows（MinGW + freeglut）
g++ -std=c++11 01.cpp -o bezier.exe -lfreeglut -lopengl32 -lglu32
```

程序运行后，可通过鼠标左键拖动红色控制点，实时观察控制多边形与曲线的变化。

六、分析与总结

实验核心是将连续参数模型映射为可渲染、可交互的离散状态。Bernstein 展开式负责提供稳定的数学定义，均匀参数采样负责将连续曲线转换为顶点序列，OpenGL 图元负责将这些顶点呈现为折线，GLUT 回调则通过修改控制点状态驱动重新求值。

实现中的主要取舍都围绕问题规模展开：固定三阶曲线选择直接 Bernstein 求值，而非通用递归算法；小批量顶点选择立即模式，而非引入现代缓冲管理；简单拖动采用整帧重采样，而非维护额外的曲线缓存。这些方案并非功能最少，而是在保证数学关系清晰的前提下避免不必要的工程复杂度。

解析验证和多组拖动结果说明，连续理论与离散实现之间保持了预期的一致性。需要进一步改进的部分主要位于离散化策略：固定参数采样并不能保证固定的几何误差，若后续面向高精度曲线输出，应引入自适应细分或弧长重参数化，而不应仅依靠增加采样数量。

七、收获

理解并掌握贝塞尔曲线的基本概念，能够理解三次贝塞尔曲线是由四个控制点得来，很容易推广到 n 个控制点。其次，结合 ai 实现了三次贝塞尔曲线的绘制和可视化控制，较为深入的了解 OpenGL 以及 C++在计算机图形学中的应用方法，收获颇丰。

附录 完整源代码

```
// Linux:   g++ -std=c++11 01.cpp -o bezier -lGL -lGLU -lglut
// Windows: g++ -std=c++11 01.cpp -o bezier.exe -lfreeglut -lopengl32 -lglu32
#include <GL/glut.h>

#include <array>
#include <cmath>
#include <cstddef>

namespace {

struct Vec2 {
    float x;
    float y;
};

constexpr int kWindowWidth = 800;
constexpr int kWindowHeight = 600;
constexpr int kSampleCount = 200;
constexpr float kPickRadius = 15.0f;
constexpr int kNoSelection = -1;

std::array<Vec2, 4> controlPoints{{
    {100.0f, 500.0f}, {250.0f, 100.0f},
    {550.0f, 100.0f}, {700.0f, 500.0f}
}};
int draggedPoint = kNoSelection;

Vec2 cubicBezier(const std::array<Vec2, 4>& points, float t) {
    const float u = 1.0f - t;
    const float b0 = u * u * u;
    const float b1 = 3.0f * u * u * t;
    const float b2 = 3.0f * u * t * t;
    const float b3 = t * t * t;

    return {
        b0 * points[0].x + b1 * points[1].x + b2 * points[2].x + b3 * points[3].x,
        b0 * points[0].y + b1 * points[1].y + b2 * points[2].y + b3 * points[3].y
    };
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(0.7f, 0.7f, 0.7f);
    glBegin(GL_LINE_STRIP);
    for (const auto& point : controlPoints) {
        glVertex2f(point.x, point.y);
    }
    glEnd();

    glColor3f(0.0f, 0.0f, 0.0f);
    glBegin(GL_LINE_STRIP);
    for (int i = 0; i <= kSampleCount; ++i) {
        const float t = static_cast<float>(i) / kSampleCount;
        const Vec2 point = cubicBezier(controlPoints, t);
        glVertex2f(point.x, point.y);
    }
    glEnd();

    glColor3f(0.9f, 0.1f, 0.1f);
    glPointSize(9.0f);
    glBegin(GL_POINTS);
    for (const auto& point : controlPoints) {
        glVertex2f(point.x, point.y);
    }
```

```
    glEnd();

    glutSwapBuffers();
}

void mouse(int button, int state, int x, int y) {
    if (button != GLUT_LEFT_BUTTON) {
        return;
    }

    if (state == GLUT_DOWN) {
        for (std::size_t i = 0; i < controlPoints.size(); ++i) {
            const auto& point = controlPoints[i];
            if (std::hypot(point.x - x, point.y - y) < kPickRadius) {
                draggedPoint = static_cast<int>(i);
                break;
            }
        }
    } else if (state == GLUT_UP) {
        draggedPoint = kNoSelection;
    }
}

void motion(int x, int y) {
    if (draggedPoint == kNoSelection) {
        return;
    }

    controlPoints[static_cast<std::size_t>(draggedPoint)] = {
        static_cast<float>(x), static_cast<float>(y)
    };
    glutPostRedisplay();
}

} // namespace

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(kWindowWidth, kWindowHeight);
    glutCreateWindow("Cubic Bezier Curve");

    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0, kWindowWidth, kWindowHeight, 0);

    glutDisplayFunc(display);
    glutMouseFunc(mouse);
    glutMotionFunc(motion);
    glutMainLoop();
    return 0;
}
```